

**MAŁOPOLSKA UCZELNIA PAŃSTWOWA
IMIENIA ROTMISTRZA WITOLDA PILECKIEGO
W OŚWIĘCIMIU**

Instytut: Nauk Inżynieryjno-Technicznych

Kierunek studiów: Informatyka

Poziom studiów: Studia pierwszego stopnia

Moduł specjalnościowy: Informatyka w przedsiębiorstwie

Forma studiów: stacjonarna

Franciszek Dębski, Patryk Wilk, Krystian Zając

nr albumu 8793, 8795, 8797

**SYSTEM MONITOROWANIA TEMPERATURY
W ARCHITEKTURZE MIKROSERWISOWEJ**

Projekt zespołowy

Prowadzący: mgr inż. Artur Pelo

Oświęcim 2026

Abstrakt

Imiona i nazwiska autorów projektu	Franciszek Dębski, Patryk Wilk, Krystian Zając
Rok urodzenia autorów projektu	2003/2004
Stopień/tytuł zawodowy, imię i nazwisko prowadzącego przedmiotu	mgr inż. Artur Pelo
Instytut/Kierunek/Poziom/ Moduł specjalnościowy	Instytut Nauk Inżynieryjno-Technicznych / Informatyka / Inżynierskie / Informatyka w przedsiębiorstwie
Data realizacji projektu	od 3.03.2026 do 9.06.2026
Nazwa przedmiotu	projekt zespołowy
Rok studiów/semestr	rok III, semestr VI

Tytuł projektu	System monitorowania temperatury w architekturze mikroserwisowej
Słowa kluczowe (max 5)	informatyka, aplikacje mobilne, bazy danych
Streszczenie (max 1400 znaków)	Celem projektu było stworzenie systemu monitorowania temperatury opartego na architekturze mikroserwisowej. Rozwiązanie integruje urządzenia IoT z nowoczesnymi aplikacjami serwerowymi i klienckimi, tworząc spójny system przesyłu danych. W skład projektu wchodzi mikrokontroler ESP32, serwer Spring Boot, baza danych MySQL oraz dwa interfejsy użytkownika: aplikacja mobilna na system Android i strona internetowa. Dane pomiarowe są przesyłane przez mikrokontroler do centralnego serwera, gdzie po uzupełnieniu o czas zapisu trafiają do bazy danych. Użytkownik może w czasie rzeczywistym śledzić aktualne odczyty oraz przeglądać pełną historię pomiarów. Obie aplikacje klienckie posiadają funkcję automatycznego odświeżania danych, co zapewnia stały dostęp do najświeższych informacji bez konieczności ręcznej obsługi systemu. Projekt potwierdził skuteczność integracji różnych technologii w jeden stabilny system komunikacji. Stworzona architektura charakteryzuje się wysoką wydajnością oraz modularnością, co pozwala na jej łatwą rozbudowę o dodatkowe funkcjonalności w przyszłości. Wszystkie założenia funkcjonalne zostały zrealizowane, a system jest gotowy do pracy w lokalnym środowisku deweloperskim.

Spis treści

Wstęp	5
1. Cel i zakres pracy	6
1.1. Cel projektu	6
1.2. Cele szczegółowe	6
1.3. Zakres pracy	6
1.4. Podział zadań i harmonogram	7
2. Analiza wymagań i projekt rozwiązania	8
2.1. Wymagania funkcjonalne	8
2.2. Wymagania нефunkcjonalne	8
2.3. Wymagania jakościowe	9
2.4. Architektura rozwiązania	9
2.5. Stos technologiczny	10
2.6. Wykorzystany sprzęt	10
2.7. Schemat płytki PCB	11
3. Opis działania systemu	13
3.1. Środowisko i struktura projektu	13
3.2. Przepływ danych oraz integracja	13
3.3. Interfejs użytkownika	13
3.4. Fragmenty kodu źródłowego	15
3.4.1. Warstwa IoT (Mikrokontroler ESP32)	15
3.4.2. Warstwa serwerowa (Spring Boot)	15
3.4.3. Warstwa mobilna (Android)	16
4. Instrukcja instalacji i użytkowania	17
4.1. Wymagania sprzętowe i programowe	17
4.2. Instrukcja instalacji	17
4.3. Instrukcja użytkowania	18
4.3.1. Obsługa aplikacji mobilnej	18
4.3.2. Obsługa interfejsu webowego	18
4.3.3. Kontrola stanu połączenia	18
5. Testy i weryfikacja	19
5.1. Testy funkcjonalne	19
5.2. Testy нефunkcjonalne	19
5.3. Wyniki testów	19
Podsumowanie	20
Netografia	21

Spis rysunków	22
Spis tabel	23
Spis listingów	25

Wstęp

Dzisiejsze rozwiązania technologiczne pozwalają na łatwe zbieranie danych z czujników i przesyłanie ich do sieci w czasie rzeczywistym. Niniejszy projekt polega na stworzeniu systemu do monitorowania temperatury, który składa się z mikrokontrolera ESP32, serwera Spring Boot oraz bazy danych MySQL. System został zaprojektowany tak, aby dane z urządzenia pomiarowego były zapisywane na serwerze i udostępniane użytkownikowi przez interfejs REST API.

Użytkownik ma dostęp do zebranych informacji za pomocą dwóch niezależnych interfejsów: strony internetowej oraz aplikacji mobilnej na system Android. Obie te aplikacje pobierają dane z tego samego źródła, co pozwala na bieżący podgląd temperatury oraz przeglądanie historii zapisów z datą i godziną. Projekt pokazuje, jak połączyć urządzenia IoT z aplikacjami webowymi i mobilnymi w ramach jednego, spójnego systemu komunikacji.

1. Cel i zakres pracy

Jasne wyznaczenie kierunków działania pozwala na precyzyjne zaplanowanie etapów pracy i uniknięcie zbędnych funkcjonalności. Dokumentacja ta definiuje granice projektu oraz to, co dokładnie zostanie dostarczone po jego zakończeniu.

1.1. Cel projektu

Głównym celem projektu było opracowanie i wdrożenie kompletnego, rozproszonego systemu pozwalającego na zdalny odczyt, archiwizację i wizualizację danych pomiarowych z fizycznego czujnika temperatury. System miał łączyć w sobie elementy sprzętowe, serwerowe oraz klienckie, tworząc spójną całość gotową do działania w warunkach lokalnych.

1.2. Cele szczegółowe

W ramach realizacji projektu wyznaczono następujące cele szczegółowe:

- zaprogramowanie mikrokontrolera ESP32 do obsługi czujnika pomiarowego oraz bezprzewodowego przesyłania danych przez Wi-Fi,
- zaimplementowanie serwera backendowego w technologii Spring Boot, pełniącego rolę centralnego węzła komunikacyjnego,
- konfiguracja i konteneryzacja relacyjnej bazy danych MySQL do trwałego przechowywania historii pomiarów,
- stworzenie natywnej aplikacji mobilnej na system Android, umożliwiającej bieżący podgląd danych,
- budowa responsywnego interfejsu webowego w formie strony internetowej do przeglądania dziennika zapisów,
- integracja wszystkich modułów za pomocą protokołu HTTP i formatu danych JSON.

1.3. Zakres pracy

Zakres pracy obejmuje następujące kroki:

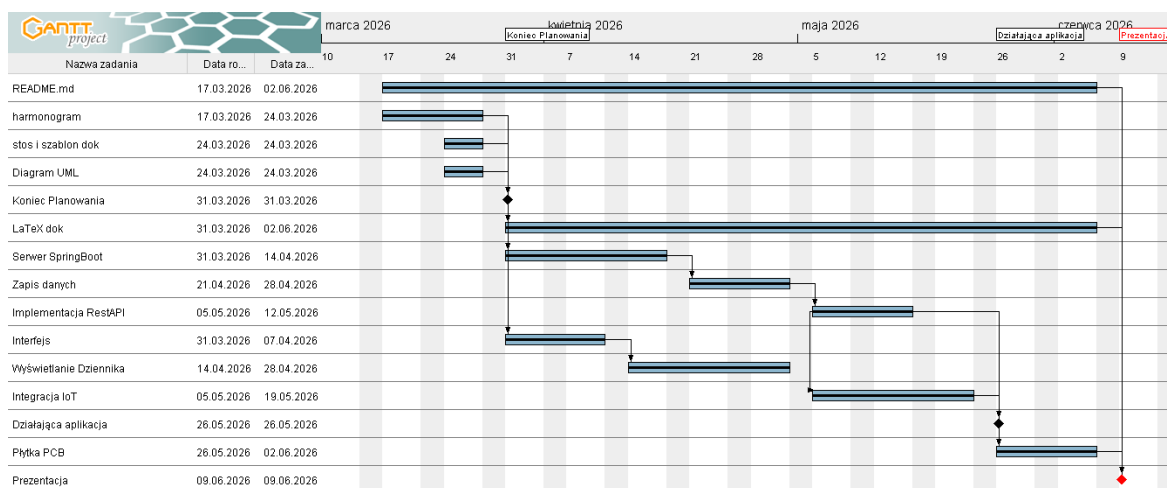
- konfiguracja warstwy sprzętowej obejmującej mikrokontroler ESP32 oraz czujnik temperatury,
- implementacja logiki serwerowej wraz z obsługą endpointów REST API,
- przygotowanie schematu bazy danych oraz skryptów łączących serwer z bazą MySQL,
- opracowanie interfejsu graficznego i logiki komunikacyjnej dla aplikacji mobilnej Android,
- przygotowanie frontendu webowego (HTML/CSS/JS) współpracującego z API serwera,
- przeprowadzenie testów funkcjonalnych i weryfikacja poprawności przepływu danych w sieci lokalnej.

1.4. Podział zadań i harmonogram

Prace nad systemem zostały podzielone między trzech członków zespołu w taki sposób, aby każdy odpowiadał za kluczową warstwę architektury mikrosерwisowej: sprzętową, serwerową lub kliencką.

- Franciszek Dębski (Programista IoT i projektant PCB): Odpowiadał za warstwę sprzętową, w tym zaprogramowanie mikrokontrolera ESP32 w języku C++ oraz zaprojektowanie modelu płytki w EasyEDA (rysunek 5 i 6). Przygotował również diagram UML (rysunek 2).
- Patryk Wilk (Główny programista): Zrealizował warstwę serwerową w Spring Boot, skonfigurował bazę danych MySQL w Dockerze oraz stworzył stronę internetową. Zarządzał harmonogramem w programie GanttProject (rysunek 1).
- Krystian Zając (Programista mobilny i Menadżer): Opracował natywną aplikację na system Android oraz przygotował pełną dokumentację techniczną w systemie LaTeX. Odpowiadał za plik README oraz prezentację końcową projektu.

Realizacja projektu została rozłożona w czasie zgodnie z ustalonym planem prac, co pozwoliło na zachowanie płynności i terminową integrację wszystkich modułów systemu. Wykres Gantta przedstawiony poniżej na rysunku 1, pokazuje szczegółowy podział na etapy, ramy czasowe poszczególnych zadań oraz kluczowe kamienie milowe projektu.



Rysunek 1. Wykres Gantta

Źródło: opracowanie własne (GanttProject)

2. Analiza wymagań i projekt rozwiązania

Analiza ta pozwala na precyzyjne określenie zadań systemu oraz zaplanowanie jego struktury tak, aby wszystkie komponenty współpracowały bezawaryjnie. Dzięki niej możliwe jest wybranie odpowiednich technologii, które najlepiej sprawdzą się w komunikacji między czujnikiem a aplikacjami końcowymi.

2.1. Wymagania funkcjonalne

Jak pokazano w tabeli 1, system musi spełniać określone funkcje. Każda z nich została przypisana do odpowiedniego modułu, co ułatwia organizację pracy i pozwala na równoległe rozwijanie poszczególnych części systemu.

Tabela 1
Wymagania funkcjonalne

ID	Opis	Priorytet
FR-001	Serwer uruchomiony i działający w oparciu o SpringBoot.	Wysoki
FR-002	Zapis danych do lokalnej bazy SQL na serwerze.	Wysoki
FR-003	Interfejs użytkownika dostępny przez przeglądarkę i aplikację mobilną.	Wysoki
FR-004	Wyświetlanie dzinnika zapisów z datą, godziną i wartościami.	Wysoki
FR-005	Integracja z urządzeniami IoT za pomocą RestAPI.	Wysoki
FR-006	Implementacja RestAPI do zarządzania danymi i interakcją.	Wysoki
FR-007	Przygotowanie dokumentacji w oparciu LaTeX. Wymagania edytorskie.	Wysoki
FR-008	Przygotowanie diagramów UML ilustrujących strukturę i zachowanie systemu.	Średni
FR-009	Dostarczenie działającej aplikacji.	Wysoki
FR-010	Utworzenie pliku README.md z opisem projektu i instrukcją instalacji.	Wysoki
FR-011	Przygotowanie modelu PCB płytki z mikrokontrolerem ESP32 Wroom.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.2. Wymagania niefunkcjonalne

Jak pokazano w tabeli 2, system musi spełniać określone standardy wydajności, bezpieczeństwa i kompatybilności.

Tabela 2
Wymagania niefunkcjonalne

ID	Opis	Priorytet
NFR-001	Czas odpowiedzi API mniejszy niż 300ms.	Wysoki
NFR-002	Dostępność systemu większa niż 99%.	Wysoki
NFR-003	Skalowalność: obsługa min. 1000 jednoczesnych użytkowników.	Wysoki
NFR-004	Bezpieczeństwo: szyfrowanie SSL/TLS dla transmisji danych.	Wysoki
NFR-005	Kompatybilność: Android 8.0+, przeglądarki wspierające HTML5.	Średni
NFR-006	Dostępność zgodna z WCAG 2.1 na poziomie AA.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.3. Wymagania jakościowe

Jak pokazano w tabeli 3, system musi spełniać określone kryteria jakościowe, aby zapewnić użytkownikowi wysoką jakość obsługi i satysfakcję.

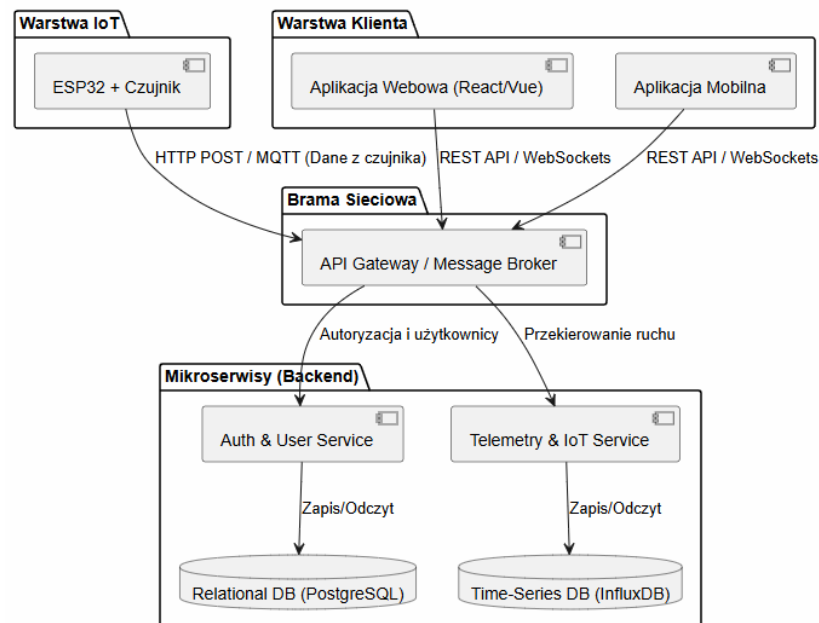
Tabela 3
Wymagania jakościowe

ID	Opis	Priorytet
QR-001	Łatwość konserwacji: modularna architektura mikroservisów.	Wysoki
QR-002	Łatwość testowania: pokrycie testami jednostkowymi i integracyjnymi większe niż 50%.	Średni
QR-003	Łatwość wdrożenia: automatyzacja procesu CI/CD.	Wysoki
QR-004	Dokumentacja: szczegółowa i aktualna dokumentacja techniczna i użytkowa.	Niski
QR-005	Czystość kodu: przestrzeganie konwencji kodowania i najlepszych praktyk.	Wysoki
QR-006	Wzorce projektowe: stosowanie odpowiednich wzorców projektowych.	Średni

Źródło: System zasobów - eStudent (Pelo, 2026)

2.4. Architektura rozwiązania

System opiera się na architekturze mikroservisowej, gdzie każda część pełni określoną funkcję jak przedstawia rysunek 2 z diageamem UML. Mikrokontroler działa jako nadawca danych, serwer jako zarządca i pośrednik, a aplikacje klienckie jako odbiorcy. Taki podział pozwala na niezależną pracę nad każdym modulem. Komunikacja odbywa się przez standardowe zapytania HTTP, co czyni system uniwersalnym i łatwym w integracji.



Rysunek 2. Diagram UML architektury systemu

Źródło: opracowanie własne

2.5. Stos technologiczny

W celu realizacji projektu wybrano następujące technologie:

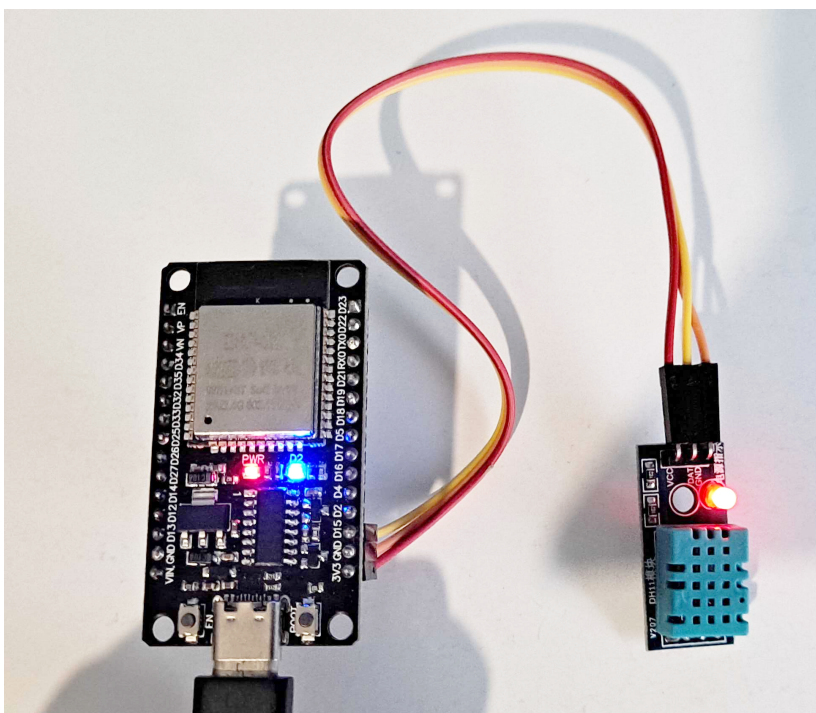
- Backend: Java, Spring Boot.
- Baza danych: MySQL (Docker/XAMPP).
- Aplikacja mobilna: Java (Android Studio).
- Aplikacja webowa: HTML, CSS, JavaScript.
- IoT: C++, ESP32 (Arduino).

2.6. Wykorzystany sprzęt

Do realizacji projektu wykorzystano następujące urządzenia:

- Mikrokontroler ESP32 Wroom.
- Czujnik temperatury DHT11.
- Komputer stacjonarny (serwer).
- Smartfon z systemem Android (klient).

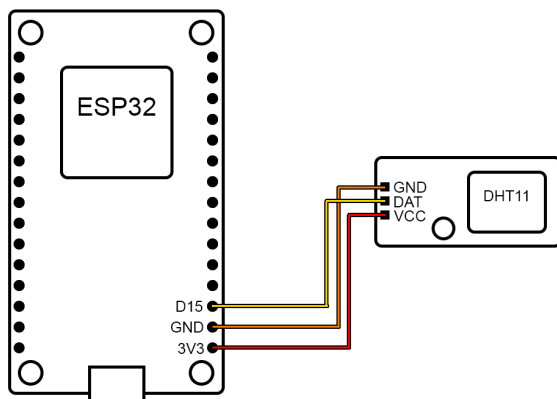
W celu realizacji projektu został wypożyczony zestaw ESP32 Wroom wraz z czujnikiem DHT11, który umożliwił odczyt danych pomiarowych i ich bezprzewodową transmisję do serwera. Jak pokazano na rysunku 3, urządzenie zostało odpowiednio podłączone według schematu połączeń przedstawionego na rysunku 4 i przygotowane do pracy w ramach systemu monitorowania temperatury.



Rysunek 3. ESP32 Wroom z czujnikiem DHT11

Źródło: opracowanie własne

Schemat połączeń elektrycznych pomiędzy mikrokontrolerem ESP32 a czujnikiem temperatury i wilgotności DHT11 przedstawiono na rysunku 4. Linia danych (DAT) czujnika została podłączona do pinu GPIO15 mikrokontrolera, natomiast zasilanie zrealizowano poprzez wyprowadzenia 3V3 oraz GND.

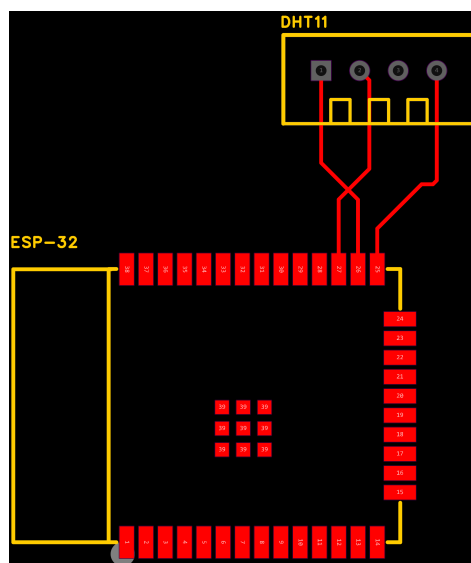


Rysunek 4. Schemat połączeń ESP32 z czujnikiem DHT11

Źródło: opracowanie własne

2.7. Schemat płytki PCB

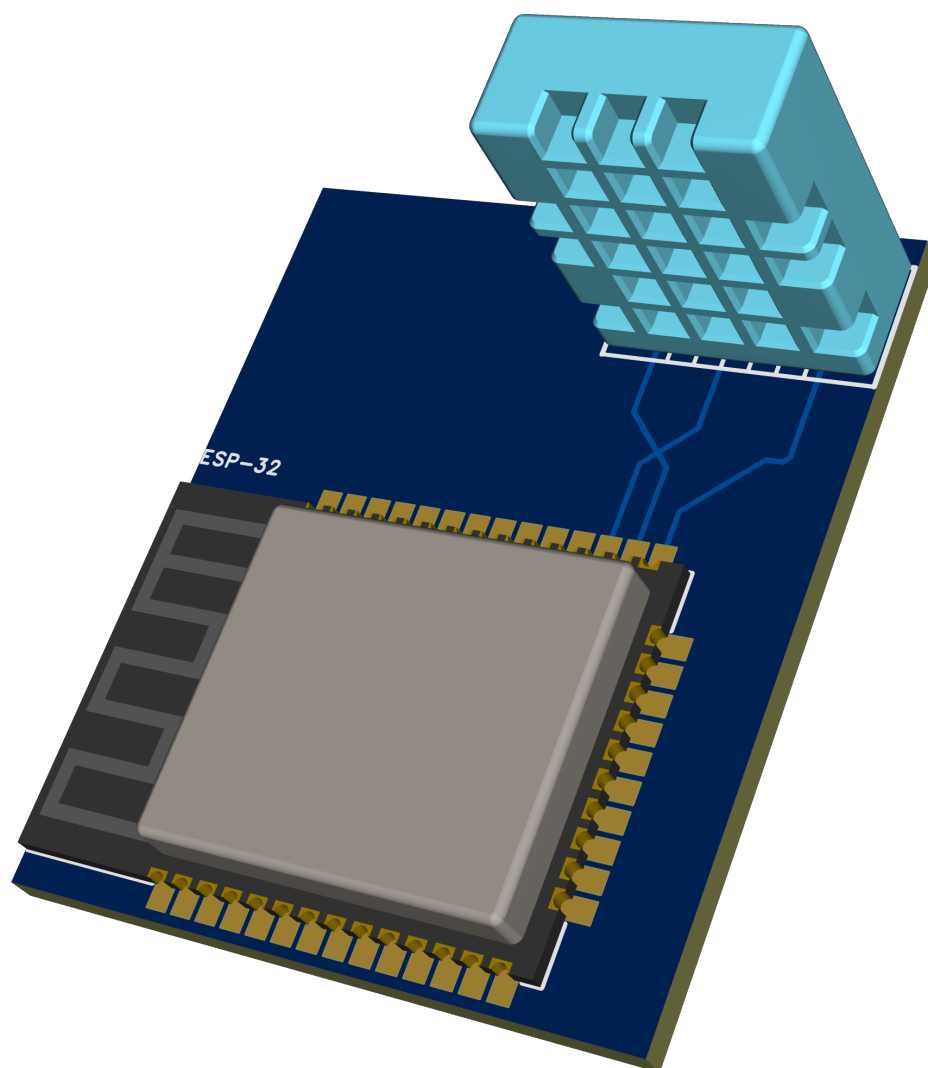
Płytką PCB została zaprojektowana w programie EasyEDA, co pozwoliło na precyzyjne rozmieszczenie elementów oraz przygotowanie modelu do ewentualnej produkcji. Schemat płytki przedstawiono na rysunku 5, gdzie widoczne są połączenia między mikrokontrolerem ESP32 a czujnikiem DHT11.



Rysunek 5. Schemat płytki PCB

Źródło: opracowanie własne (EasyEDA)

Sporządzona została też wizualizacja 3D płytki PCB, która umożliwia lepsze zrozumienie rozmieszczenia komponentów oraz ich wzajemnych połączeń. Jak pokazano na rysunku 6, model 3D przedstawia szczegółowy układ elementów na płytce, co jest istotne dla dalszych etapów produkcji i montażu.



Rysunek 6. Wizualizacja przestrzenna płytki PCB

Źródło: opracowanie własne (EasyEDA)

3. Opis działania systemu

Szczegółowe przedstawienie mechanizmów działania systemu tłumaczy sposób, w jaki dane przemieszczają się między urządzeniami. Ułatwia to zrozumienie logiki aplikacji oraz technicznych aspektów integracji poszczególnych modułów.

3.1. Środowisko i struktura projektu

Podczas prac deweloperskich wykorzystano środowiska IntelliJ IDEA oraz Android Studio. Struktura projektu jest podzielona na niezależne moduły: projekt serwerowy, projekt mobilny oraz pliki frontendu webowego. Baza danych MySQL pracuje jako osobna usługa, co symuluje rzeczywiste warunki pracy systemów sieciowych.

3.2. Przepływ danych oraz integracja

Dane przepływają w systemie w sposób liniowy. Mikrokontroler ESP32 po odczytaniu temperatury formuje paczkę JSON i wysyła ją na endpoint serwera. Serwer Spring Boot przetwarza tę paczkę, dodaje do niej datę i godzinę systemową, po czym wysyła polecenie zapisu do bazy danych. Aplikacje mobilna i webowa w stałych odstępach czasu (co 10 sekund) odpytują serwer o najnowsze dane. Jeśli w bazie pojawiły się nowe rekordy, interfejsy użytkownika są natychmiastowo aktualizowane.

3.3. Interfejs użytkownika

Interfejs webowy, przedstawiony na rysunku 7, został zaprojektowany z myślą o przejrzystości i łatwości nawigacji. Głównym elementem jest tabela, która wyświetla wszystkie zarejestrowane pomiary temperatury wraz z datą i godziną. Dzięki zastosowaniu prostego układu, użytkownik może szybko znaleźć interesujące go dane oraz przeanalizować historię pomiarów.

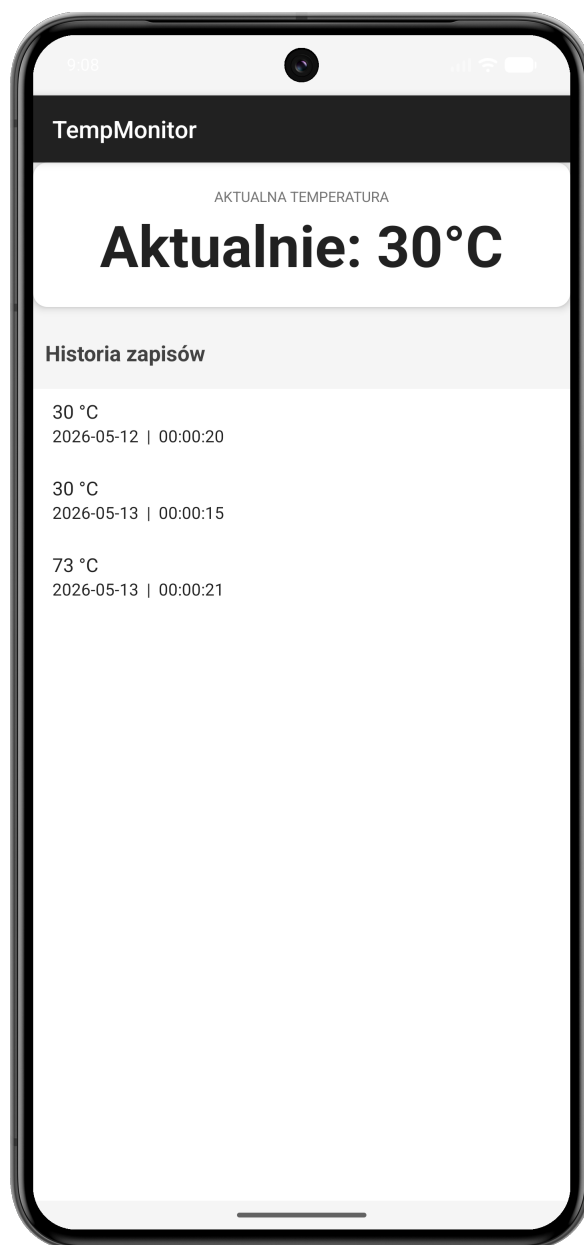


Monitor Temperatury		
Ostatnia aktualizacja: 20:56:14		
Data	Godzina	Temperatura
2026-05-18	18:55:43	25 °C

Rysunek 7. Interfejs webowy

Źródło: opracowanie własne

Interfejs mobilny, przedstawiony na rysunku 8, został zaprojektowany z myślą o prostocie i funkcjonalności. Głównym elementem jest karta z aktualną temperaturą, a poniżej znajduje się lista wszystkich pomiarów, która jest automatycznie odświeżana co 10 sekund, zapewniając użytkownikowi dostęp do najnowszych danych bez konieczności ręcznego odświeżania aplikacji.



Rysunek 8. Interfejs mobilny

Źródło: opracowanie własne (Android Studio)

3.4. Fragmenty kodu źródłowego

Poniżej przedstawiono fragmenty kodu źródłowego wraz z opisami, które ilustrują kluczowe elementy działania systemu. Przedstawione przykłady dotyczą głównie warstwy IoT, serwerowej oraz mobilnej, co pozwala na zrozumienie mechanizmów komunikacji i przetwarzania danych w całym systemie.

3.4.1. Warstwa IoT (Mikrokontroler ESP32)

Mikrokontroler został skonfigurowany w trybie punktu dostępowego, co oznacza, że tworzy on własną sieć Wi-Fi o nazwie „Wifi XPP”. Komputer z serwerem musi być podłączony do tej sieci, aby komunikacja była możliwa. Jak zaprezentowano w listingu 1, urządzenie w pętli co 60 sekund odczytuje temperaturę z czujnika DHT11 i wysyła ją metodą POST w formacie JSON pod stały adres IP serwera.

Listing 1. Fragment kodu odpowiedzialny za wysyłanie danych

```
1 void wyslijDane(float temp) {
2   HTTPClient http;
3   http.begin("http://192.168.4.2:8080/api/pomiary/arduino");
4   http.addHeader("Content-Type", "application/json");
5   String httpRequestData = "{\"temperatura\":\"" + String(temp) + "\"}";
6
7   int httpResponseCode = http.POST(httpRequestData);
8
9   if (httpResponseCode > 0) {
10    Serial.printf("Sukces, kod: %d\n", httpResponseCode);
11  } else {
12    Serial.printf("Błąd połączenia: %s\n", http.errorToString(
13      httpResponseCode).c_str());
14  }
15  http.end();
16 }
```

3.4.2. Warstwa serwerowa (Spring Boot)

Serwer pełni rolę pośrednika i archiwizatora. Jego zadaniem jest odebranie wartości liczbowej od ESP32 i dopisanie do niej brakujących informacji, takich jak aktualna data i godzina, ponieważ mikrokontroler nie posiada własnego zegara czasu rzeczywistego.

Listing 2. Fragment kodu odpowiedzialny za odbieranie danych

```
1 @PostMapping("/arduino")
2 public ResponseEntity<String> receiveFromArduino(@RequestBody Dane data) {
3
4   data.setData(java.time.LocalDate.now());
5   data.setGodzina(java.time.LocalTime.now());
6
7   repository.save(data);
8
9   return ResponseEntity.ok("Dane zapisane pomyślnie");
10 }
```

3.4.3. Warstwa mobilna (Android)

Aplikacja mobilna działa w trybie odczytu. Aby zapewnić użytkownikowi świeże informacje bez konieczności ręcznego odświeżania, zastosowano mechanizm pętli czasowej, która co 10 sekund wysyła zapytanie GET do serwera.

Listing 3. Fragment kodu odpowiedzialny za odbieranie danych

```
1 refreshRunnable = new Runnable() {  
2     @Override  
3     public void run() {  
4         // Metoda pobierająca listę wszystkich pomiarów z serwera  
5         pobierzDaneZSerwera();  
6  
7         // Planowanie kolejnego wykonania kodu za 10 000 milisekund  
8         handler.postDelayed(this, 10000);  
9     }  
10 };
```

Zastosowanie obiektu Handler pozwala na cykliczne wykonywanie zadania w tle. Metoda pobierzDaneZSerwera wykorzystuje bibliotekę Retrofit do pobrania listy obiektów, która następnie jest przekazywana do adaptera i wyświetlana na ekranie telefonu w formie chronologicznego dziennika.

4. Instrukcja instalacji i użytkowania

Dostarczenie niezbędnych informacji technicznych pozwala na poprawne uruchomienie i obsługę gotowego systemu przez użytkownika końcowego. Dzięki temu proces wdrażania aplikacji w nowym środowisku staje się prosty i przewidywalny.

4.1. Wymagania sprzętowe i programowe

Aby system mógł pracować wydajnie i bez opóźnień, urządzenia wykorzystywane w projekcie muszą spełniać określone parametry techniczne. Poniższe zestawienie przedstawia niezbędne minimum sprzętowe oraz oprogramowanie wymagane do poprawnej kompilacji i działania wszystkich modułów:

- Komputer stacjonarny lub laptop: Procesor wspierający virtualizację, minimum 8 GB pamięci RAM oraz system Windows 10/11, Linux lub macOS.
- Urządzenie mobilne: Smartfon lub tablet z systemem Android w wersji minimum 8.0.
- Mikrokontroler: Układ ESP32 Wroom wraz z podłączonym czujnikiem temperatury DHT11.
- Oprogramowanie systemowe: Docker Desktop, środowisko Java (JDK 24) oraz przeglądarka internetowa wspierająca standard HTML5.

4.2. Instrukcja instalacji

Proces instalacji systemu został podzielony na etapy, które należy wykonywać w określonej kolejności. Jest to kluczowe, ponieważ aplikacje klienckie oraz mikrokontroler wymagają aktywnego i skonfigurowanego serwera do poprawnego działania.

- Uruchomienie serwera (Docker): Należy uruchomić kontener MySQL i Spring Boot za pomocą Dockera. W tym celu uruchom program Docker Desktop, otwórz folder "container things", a następnie w terminalu i wpisz komendę "docker-compose up – build".
- Programowanie mikrokontrolera: Korzystając z Arduino IDE, należy wgrać kod do układu ESP32. W kodzie źródłowym należy wcześniej wpisać nazwę i hasło lokalnej sieci Wi-Fi oraz adres IP komputera, na którym działa serwer.
- Instalacja aplikacji mobilnej: Plik aplikacji należy skompilować w Android Studio i zainstalować na telefonie. W ustawieniach połączenia (klasa ApiClient/MainActivity) musi znajdować się ten sam adres IP serwera, co w przypadku ESP32.
- Uruchomienie interfejsu webowego: Plik index.html należy otworzyć w dowolnej nowoczesnej przeglądarce internetowej na komputerze podłączonym do tej samej sieci.

4.3. Instrukcja użytkowania

Korzystanie z gotowego systemu jest intuicyjne, ponieważ większość procesów związanych z przesyłaniem i wyświetlaniem danych odbywa się automatycznie. Po prawidłowym uruchomieniu wszystkich modułów, użytkownik może monitorować temperaturę w sposób opisany poniżej.

4.3.1. Obsługa aplikacji mobilnej

Po uruchomieniu aplikacji na smartfonie, główny ekran natychmiast wyświetla ostatni zarejestrowany pomiar w wyróżnionej karcie na górze. Poniżej znajduje się chronologiczna lista wszystkich zapisów. Aplikacja sama odpytuje serwer o nowe dane co 10 sekund, więc użytkownik nie musi ręcznie odświeżać widoku, aby zobaczyć najnowsze zmiany.

4.3.2. Obsługa interfejsu webowego

W celu przejrzania danych na komputerze wystarczy otworzyć przygotowany plik strony w dowolnej przeglądarce internetowej. Wyświetli on czytelną tabelę z kompletną historią zapisów, co pozwala na wygodną analizę danych archiwalnych.

4.3.3. Kontrola stanu połączenia

System został zaprojektowany tak, aby na bieżąco informować o ewentualnych problemach technicznych. Jeśli telefon straci połączenie z serwerem lub serwer zostanie wyłączony, na ekranie aplikacji mobilnej pojawi się komunikat informujący o braku łączności, co ułatwia szybkie zdiagnozowanie problemu z siecią Wi-Fi.

5. Testy i weryfikacja

Przeprowadzenie weryfikacji potwierdza, że system został wykonany zgodnie z planem i spełnia wszystkie założone wymagania techniczne. Dokumentacja ta stanowi dowód na poprawność działania i niezawodność całej stworzonej architektury.

5.1. Testy funkcjonalne

Testy wykazały, że komunikacja między mikrokontrolerem a serwerem przebiega bez zakłóceń, a dane są poprawnie zapisywane w bazie MySQL. Sprawdzono również aplikacje klienckie, potwierdzając, że zarówno strona internetowa, jak i aplikacja mobilna wyświetlają aktualne dane oraz pełną historię pomiarów w odpowiedniej kolejności chronologicznej.

5.2. Testy нефunkcjonalne

Weryfikacja wydajności potwierdziła, że średni czas odpowiedzi API wynosi około 50 ms, co znacznie przewyższa założony limit 300 ms. Przetestowano także stabilność automatycznego odświeżania danych co 10 sekund oraz responsywność interfejsu webowego, który poprawnie dopasowuje się do różnych rozdzielczości ekranu.

5.3. Wyniki testów

System pomyślnie przeszedł wszystkie scenariusze testowe i nie wykazuje błędów w przepływie informacji. Aplikacje poprawnie reagują na brak połączenia z serwerem, wyświetlając stosowne komunikaty, co potwierdza, że stworzona architektura jest stabilna i gotowa do pracy w środowisku lokalnym.

Podsumowanie

Realizacja projektu pozwoliła na stworzenie stabilnego systemu, który poprawnie integruje mikrokontroler z aplikacjami mobilną i webową. Wszystkie założone cele zostały osiągnięte, a dane pomiarowe są przesyłane i archiwizowane w bazie danych bez opóźnień. System udowodnił swoją użyteczność w warunkach lokalnych, oferując użytkownikowi stały i automatyczny dostęp do informacji o temperaturze.

W przyszłości projekt można rozbudować o system powiadomień alarmowych wysyłanych na telefon po przekroczeniu bezpiecznych progów temperatury. Warto również rozważyć dodanie nowych czujników, np. wilgotności powietrza, oraz przeniesienie serwera do chmury, co pozwoliłoby na dostęp do danych z dowolnego miejsca na świecie. Tak przygotowana architektura stanowi solidną bazę do dalszego rozwoju w stronę systemów inteligentnego domu.

Netografia

Pelo, Artur (2026). *System zasobów - eStudent*. URL: <https://pelo.com.pl/> (dostęp 19.05.2026).

Spis rysunków

<i>Rysunek 1.</i> Wykres Gantta	7
<i>Rysunek 2.</i> Diagram UML architektury systemu	9
<i>Rysunek 3.</i> ESP32 Wroom z czujnikiem DHT11	10
<i>Rysunek 4.</i> Schemat połączeń ESP32 z czujnikiem DHT11	11
<i>Rysunek 5.</i> Schemat płytki PCB	11
<i>Rysunek 6.</i> Wizualizacja przestrzenna płytki PCB	12
<i>Rysunek 7.</i> Interfejs webowy	13
<i>Rysunek 8.</i> Interfejs mobilny	14

Spis tabel

Tabela 1. Wymagania funkcjonalne	8
Tabela 2. Wymagania нефункционалне	8
Tabela 3. Wymagania jakościowe	9

Spis listingów

Listing 1. Fragment kodu odpowiedzialny za wysyłanie danych	15
Listing 2. Fragment kodu odpowiedzialny za odbieranie danych	15
Listing 3. Fragment kodu odpowiedzialny za odbieranie danych	16